

It's Sunday morning and the conference's official app — the one everyone uses to check the schedule from their phone — starts acting up. It's not crashing, which would almost be better: it's just **painfully slow**.

The problem came to light when someone, searching for **Py-thagoras'** talk about the new profiler, typed "py" into the search bar... and sat there watching the loading spinner for a suspiciously long time for something that should be instant. With over two thousand talk titles, workshops, and lightning talks in the database, the search bar is comparing the typed text against **every single word, letter by letter, every time you press a key**.

Ohhh nooo, this is way too slow.

They call you. Again.

—"We need to show, as the person types, how many talks have a title starting with whatever they've typed so far. And it needs to be fast, because people around here type *fast*."

You look at the current code: for every letter someone types, it scans the entire list of titles, comparing them one by one. With thousands of titles and thousands of searches per second, that's not going to hold up. You need a solution that's **much faster than brute force**.

Problem Statement

You are given a list of n words (talk titles, lowercase, no spaces, each treated as a single word). You are then given q queries, each one a prefix. For each query, you must report **how many words in the list start with that prefix** (a word counts as a prefix of itself).

Comparing every query against every word, letter by letter, is too slow for this problem's limits: you need a solution whose cost doesn't depend on scanning every word on every single query.

Input Format

- The first line contains an integer n , the number of words.
- Each of the following n lines contains one word (lowercase letters only, no spaces).
- The next line contains an integer q , the number of queries.
- Each of the following q lines contains a prefix (lowercase letters only, non-empty).

Constraints

```
1 ≤ n ≤ 10^5
1 ≤ q ≤ 10^5
1 ≤ length of each word ≤ 50
1 ≤ length of each prefix ≤ 50
```

Output Format

For each query, print on its own line a single integer: the number of words that start with that prefix.

Sample Input 0

```
7
python
pytest
pyplot
pycon
pygame
java
javascript
6
py
pyt
java
pyg
j
python
```

Sample Output 0

```
5
2
2
1
2
1
```

Explanation 0

- "py" is a prefix of **python**, **pytest**, **pyplot**, **pycon**, **pygame** → 5
- "pyt" is a prefix of **python**, **pytest** → 2
- "java" is a prefix of **java**, **javascript** → 2
- "pyg" is a prefix of **pygame** → 1
- "j" is a prefix of **java**, **javascript** → 2

- "python" is a prefix of itself → 1